Layers as lenses: how a two-hidden-layer ReLU network learns an oblique decision tree, step by step

A handout for new collaborators

Harish G. and collaborators (`layer-lenses` repository, October 2026)

Abstract

We study how gradient descent builds a hierarchy of features. The task is a depth-5 oblique decision tree (ODT) on the unit sphere in $\mathbb{R}^{100}$: 15 internal nodes with orthonormal normals route each input to one of 16 label nodes, where the label is the sign of a sixteenth normal. The network is a 62–60–1 ReLU network without biases. The central idea is that every first-layer neuron sees the data *through a lens*: the active region of the second-layer units that read it. Its gradient is the gradient of a linear classifier on data reweighted by that lens, so a neuron learns whatever direction the lens makes informative. We design an initialisation in which no first-layer weight vector contains any internal normal and all the information about the tree sits in the wiring of the second layer, and we show that plain SGD then discovers the 15 internal normals level by level, builds 32 leaf-cell detectors and reaches a test error below 0.1 %. A symmetrised gradient turns the same runs into a simulation of population gradient flow, which separates the deterministic mechanism from symmetry breaking by sampling noise. Every number and figure comes from eight comparable runs stored in the repository.

1 The output of a ReLU network as a sum over paths

Consider a network with two hidden layers and no biases, $f(x) = W_{\text{out}}\,\mathrm{ReLU}\big(W_2\,\mathrm{ReLU}(W_1 x)\big)$, with $W_1 \in \mathbb{R}^{62\times 100}$, $W_2 \in \mathbb{R}^{60\times 62}$, $W_{\text{out}} \in \mathbb{R}^{1\times 60}$. We call the 62 first-layer units *neurons* and write $w_r \in \mathbb{R}^{100}$ for the weight vector of neuron r (a row of W_1); the 60 second-layer units are *units*. Let $g_r(x) = \mathbf{1}[w_r \cdot x > 0]$ and $g_u(x) = \mathbf{1}[(W_2\,\mathrm{ReLU}(W_1 x))_u > 0]$ be the gates. Because $\mathrm{ReLU}(z) = g(z)\,z$,

$$f(x) \;=\; \sum_{u=1}^{60}\sum_{r=1}^{62} W_{\text{out},u}\,g_u(x)\,W_{2,ur}\,g_r(x)\,(w_r\cdot x) \;=\; \sum_{(u,r)} \underbrace{W_{\text{out},u}W_{2,ur}}_{\text{path weight}}\;\underbrace{g_u(x)g_r(x)}_{\text{path gate}}\,(w_r\cdot x). \tag{1}$$

The output is a sum over *paths* (u, r); each path is a linear function of x switched on inside a region of input space, the intersection of the active regions of its neuron and its unit. The function is piecewise linear and positively homogeneous of degree 3 in the parameters.

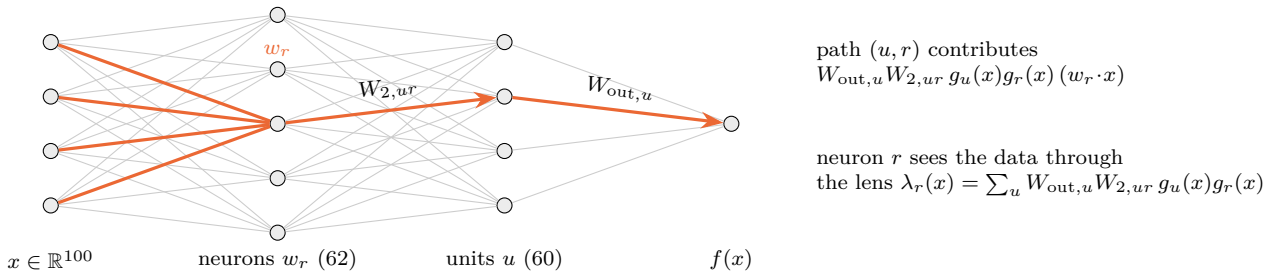


Figure 1: The path decomposition (1). A neuron's gradient is a linear-classifier gradient on data reweighted by its lens λ_r.

Gradient of a neuron. For the logistic loss $\ell(y, f) = \log(1 + e^{-yf})$ the population gradient with respect to w_r is

$$-\nabla_{w_r}\,\mathbb{E}\,\ell \;=\; \mathbb{E}\big[\sigma(-yf(x))\,y\,\lambda_r(x)\,x\big], \qquad \lambda_r(x) = \sum_u W_{\text{out},u}W_{2,ur}\,g_u(x)\,g_r(x). \tag{2}$$

This is the gradient of a *linear* classifier $w_r \cdot x$ on the data reweighted by $\sigma(-yf)\lambda_r(x)$. The weight λ_r is the neuron's lens: it is set by the units that read the neuron (their readout, their weight on the neuron, their active region) and by the neuron's own gate. At initialisation $f \approx 0$ and $\sigma(-yf) \approx \frac{1}{2}$, so a neuron moves along $\sum_u W_{\text{out},u}W_{2,ur}\,\mathbb{E}[g_u g_r\,y\,x]$. Everything in this handout follows from asking what $\mathbb{E}[g_u g_r\,y\,x]$ is for the ODT data and for the units we build.

2 The data: a depth-5 oblique decision tree

Inputs are uniform on the unit sphere in $\mathbb{R}^{100}$. A fixed set of 31 orthonormal vectors $u_0, \dots, u_{30}$ (drawn once, tree seed 5178) is arranged as a complete binary tree in heap order (Figure 2): node j has children $2j+1$ and $2j+2$. An input descends from the root: at internal node j it goes to $2j+1$ if $u_j \cdot x < 0$ and to $2j+2$ otherwise. After four decisions it reaches a label node $m \in \{15, \dots, 30\}$ and the label is $y = \mathrm{sign}(u_m \cdot x)$. We call the 15 internal normals *routing normals* and the 16 others *leaf normals*; the region of inputs that reaches label node m is the *leaf cell* of m, each of probability $1/16$. The label is noiseless, so the Bayes error is 0. The task is invariant under every tree flip (negate u_j and exchange the two subtrees below j), under rotations of the 69-dimensional orthogonal complement of the normals, and anti-invariant under negating all 16 leaf normals (which negates y). All runs use 10^7 training points per epoch in minibatches of 10^4 (1000 steps per epoch) and a fixed test set of 2×10^5 points.

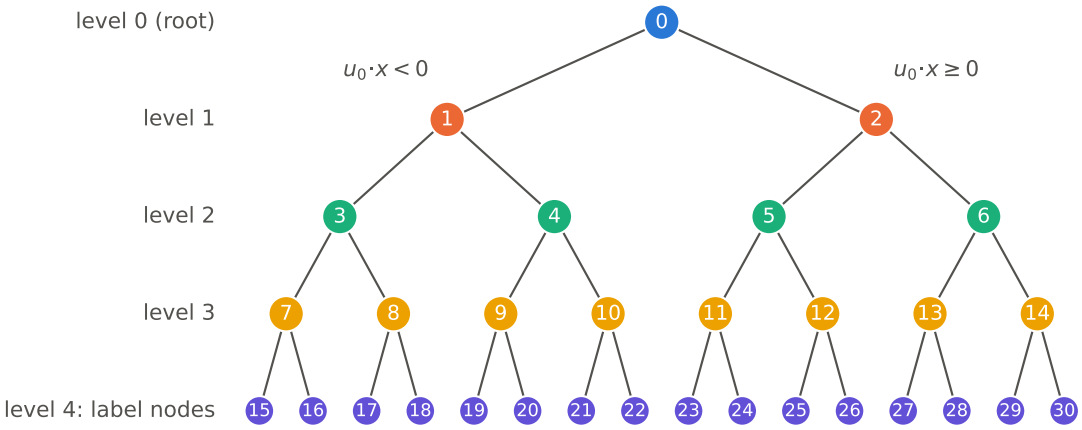


Figure 2: The tree. Internal nodes 0–14 route with the routing normals (colours mark the levels used throughout: level 0, level 1, level 2, level 3); label nodes 15–30 (violet) carry the leaf normals that set the label.

What a linear model sees. The best linear classifier is a useful reference. By strict convexity and the symmetries above, its population optimum is exactly $c\,v$ with $v = \sum_{m=15}^{30} u_m$, the sum of the leaf normals: every routing component and every off-span component is forced to zero. Numerically $c \approx 1.05$, the loss is 0.672 (against $\log 2 = 0.693$) and the error is 41.9 %. The direction v matters below: it is the direction a neuron converges to when its lens is uninformative, and the designed cues are built around it.

Flow time. Trajectories are plotted against the *flow time* $\tau = \sum_t \eta_t$, the cumulative step size, rather than the epoch, so that runs with different schedules are comparable and so that the picture is that of gradient flow $\dot{\theta} = -\nabla L$ sampled at times τ. At the full step $\eta = 0.1$ one epoch (1000 steps) is $\tau = 100$; the plots use units of 1000, so one unit is ten epochs at the full step.

3 The network and the two designed initialisations

Architecture and schedule. 62 first-layer neurons, 60 second-layer units, one output, no biases, plain SGD on the logistic loss. The step is constant at 0.1 for the first 1000 epochs, then follows one cosine down to 0.005 over the remaining epochs (2500 epochs in all). In the *two-phase* protocol the cue neurons (below) are frozen for the first 1000 epochs (flow time 100 in the plots' units) and everything is free afterwards; in the *one-phase* protocol nothing is frozen. The first-layer neurons have two roles fixed at initialisation:

- **30 probe neurons**, one per (internal node j, pole $\pm$). Each weight vector is $\sqrt{2}$ times a unit vector in the orthogonal complement of all 31 normals, the 30 directions being mutually orthogonal. A probe therefore starts with *no component on any tree normal*: it is pure noise with respect to the task.
- **32 cue neurons**, one per (label node m, pole $\pm$): $w = \pm\sqrt{2}\,(v + \beta u_m)/\|v + \beta u_m\|$ with $\beta = 1$. A cue is the linear-model direction v with a bias towards its own leaf normal; its cosine with u_m is 0.46 and with every other leaf normal 0.23.

The second layer has four units per internal node j (60 units for 15 nodes), with a block sign b and a path sign $s \in \{\pm 1\}$, the four types $(b, s) \in \{(+,+),(+,-),(-,-),(-,+)\}$. Each unit:

- **vetoes one probe of its node**: weight -1 on the probe of pole sign(bs). With ReLU this does nothing until the probe carries a direction; it is where the probe's gradient enters.
- **reads the cues with a side contrast** (*design 60*, Figure 3, left): weight $s \cdot \text{pole} \cdot c_l$ on the cues of the leaves under j on one side (the "open" side) and $-s \cdot \text{pole} \cdot c_l$ on the other side, nothing outside the subtree, with $c_l = 2^{-(5-l)/2}$ for a node of level l $(0.18, 0.25, 0.35, 0.5)$. The readout is $s \times 0.8165$ and its learning rate is multiplied by 80 (this replaces a third layer used in earlier work).
- **or reads them with a weak majority** (*design 70*, Figure 3, right): 0.2 on the open side, 0.1 on the vetoed side, 0.1 outside the subtree (times $s \cdot$ pole), an extra $+1$ link to the partner probe of the vetoed one, the same readout, no readout speed-up.

The designs are named after the runs that introduced them. The whole family is parametrised by thirteen scalars $\alpha_1, \ldots, \alpha_{13}$ (neuron norms, cue bias, the three cue weights, the two probe links, the two readout sizes, two learning-rate scales, the cues' in-span fraction) in `layer_lenses.alpha_init`; the first layer is the same in both designs.

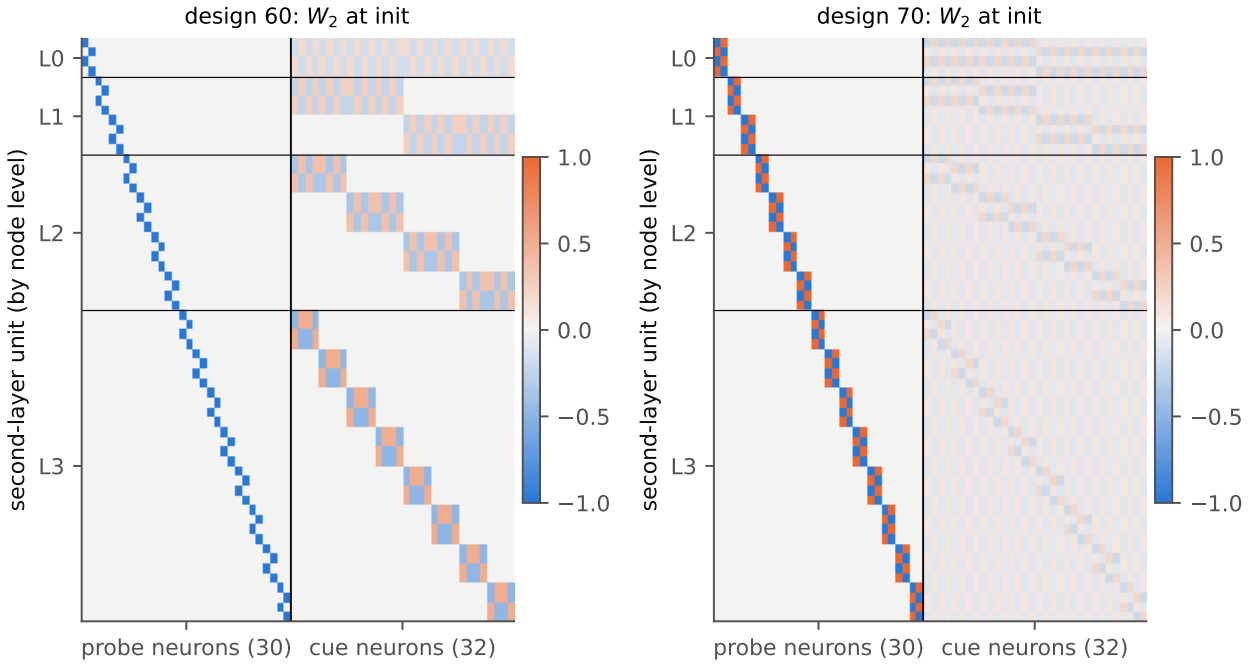


Figure 3: W_2 at initialisation (rows: the 60 units grouped by the level of their node; columns: the 62 first-layer neurons). Left, design 60: the diagonal of -1 entries on the probe columns is the vetoes; the block pattern on the cue columns is the side contrast, with magnitude growing with the level. Right, design 70: vetoes -1 and partner links $+1$ on the probe columns, weak majority weights 0.1–0.2 on the cue columns.

Why these initialisations are interesting. Nothing in the first layer knows the 15 routing normals: the probes are random off-span directions, and the cues only know their own leaf. The tree is encoded in *which cues each unit reads and with which sign*. By (2), the probe vetoed by unit u receives the push $W_{\text{out},u} W_{2,up}\, \mathbb{E}[g_u\, y\, x]$ once it has any direction at all, and $\mathbb{E}[g_u\, y\, x]$ restricted to a leaf cell m has a component along every routing normal a above m, with the sign of the side of a on which m lies. A unit whose cue weights favour the leaves on one side of node j has an active region biased to that side, so the push on its probe along u_j is $\propto \sum_{m \text{ under } j} \text{side}_j(m)\, p_u(m)$, where $p_u(m)$ is the probability that the unit is active on cell m. Design 60's side contrast makes this sum large; a unit that reads both sides equally pushes nothing along u_j, and design 70's weak majority sits in between. The same formula gives a push along every ancestor of j, which is why the probes acquire a transient ancestor component. Both inits are exact fixed points of the tree symmetries, which is what makes Section 6 possible, and they are small enough that every quantity in the trajectory can be read off.

4 Results: the eight comparable runs

Table 1 collects the eight comparable runs: two designs, plain SGD or the symmetrised flow of Section 6, two phases or one. Figure 4 shows the reference run in full, Figure 5 the SGD run on design 70.

run	design, optimiser, phases	probe capture: epoch (flow time) L0	L1	L2	L3	cues rotated epoch	loss / error / nodes at the end
123	60, SGD, two	10 (1.0)	39 (3.9)	22 (2.2)	31 (3.1)	1052	0.0042 / 0.08 % / 31
125	60, symmetrised, two	10 (1.0)	39 (3.9)	22 (2.2)	30 (3.0)	1129	0.0026 / 0.01 % / 31
124	70, SGD, two	107 (10.7)	– (–)	– (–)	– (–)	–	0.0577 / 1.45 % / 23
126	70, symmetrised, two	96 (9.6)	370 (37.0)	310 (31.0)	363 (36.3)	1104	0.0196 / 0.48 % / 31
127	60, SGD, one	9 (0.9)	16 (1.6)	26 (2.6)	– (–)	–	0.0060 / 0.19 % / 31
129	60, symmetrised, one	9 (0.9)	41 (4.1)	25 (2.5)	– (–)	–	0.0373 / 0.43 % / 15
128	70, SGD, one	34 (3.4)	601 (60.1)	164 (16.4)	– (–)	–	0.0312 / 0.83 % / 28
130	70, symmetrised, one	33 (3.3)	189 (18.9)	127 (12.7)	– (–)	–	0.0619 / 0.81 % / 15

Table 1: The eight comparable runs (same first layer, same data order, same schedule). Capture = first epoch at which every probe of the level has $|\cos| > 0.95$ with its own routing normal, with the flow time in units of 1000 in brackets; "cues rotated" = first epoch at which all 32 cues have $|\cos| > 0.95$ with their own leaf normal; "nodes" = normals covered by some neuron at the end (31 = all). A dash means never within the run.

Design 60. Under SGD (run 123) the probes are captured level by level within the first few units of flow time (root first; the level-1 probes last, because their units compete with the root's for the same cue readings), the loss then sits on a plateau of about 0.105 while the cues are frozen, and the release of the cues at flow time 100 is followed by their rotation onto the leaf normals and a fall of the loss to 0.0042 (test error 0.08 %), with all 31 normals covered. The symmetrised run (125) does the same at the same flow times and ends lower (0.0026, 0.01 %): on this design the noise does not matter.

Design 70. Under SGD (run 124, Figure 5) only the root is captured cleanly; the probes of levels 1–3 end with $|\cos| \approx 0.4$–0.5 with their own node because several go to the root or to an ancestor instead, and the run ends at loss 0.0577 with 23 of 31 normals covered. The symmetrised run from the same init (126) captures all four levels and covers all 31 normals (Section 6).

5 How the tree is represented in the trained network

Figure 6 shows run 125 at the end, Figure 7 the readouts along the run, and Figure 8 one unit. The representation is explicit:

1. **The first layer is a feature bank of the normals.** Each probe has become $\pm u_j$ for its node (the two probes of a node carry the two poles), with mean norms 25.8, 17.6, 13.2, 10.2 by level; each cue has rotated to $\pm u_m$ with norm ≈ 3.3. The off-span part of every weight vector has gone. A neuron of the form $\|w\|\,\mathrm{ReLU}(\pm u_j \cdot x)$ is a half-space detector for one side of one node.
2. **Units are** ReLU(**background − vetoes**). A level-3 unit (node j of level 3, leaf m below it) has weight ≈ -7.2 on its own probe and $-6.1, -5.6, -5.8$ on one probe of each of its three ancestors (Figure 8: unit 28 of node 7 vetoes u_7 and u_3, u_1, u_0), a veto of -4.8 on the opposite-pole cue of its leaf (cue $(15, -1)$), and small positive weights on the other cues. The vetoes switch the unit off whenever x is on the wrong side of any ancestor, so the unit is active on 3.3 % of the inputs and 99 % of its active inputs lie in a single leaf cell: it is a *leaf-cell detector*, and inside the cell its output follows $\mathrm{sign}(u_m \cdot x)$ through the cues.
3. **The level-3 units carry the function.** Their readouts are ± 118.2 on average against 21.7 at level 0 (Figure 7); removing the 28 units of levels 0–2 raises the test error only from 0.01 to 0.14 %, while removing the 32 level-3 units raises it to 25.16 %. On each leaf cell the logit is a sum of $\mathrm{ReLU}(\pm u_m \cdot x)$ terms with the right signs, which is the ODT itself. The units of levels 0–2 are the *scaffold*: they are the units that pushed the probes of their own node (a level-3 unit has no weight on an ancestor's probe at init, so it cannot create the ancestor's feature, only use it once it exists), and they become dispensable afterwards.

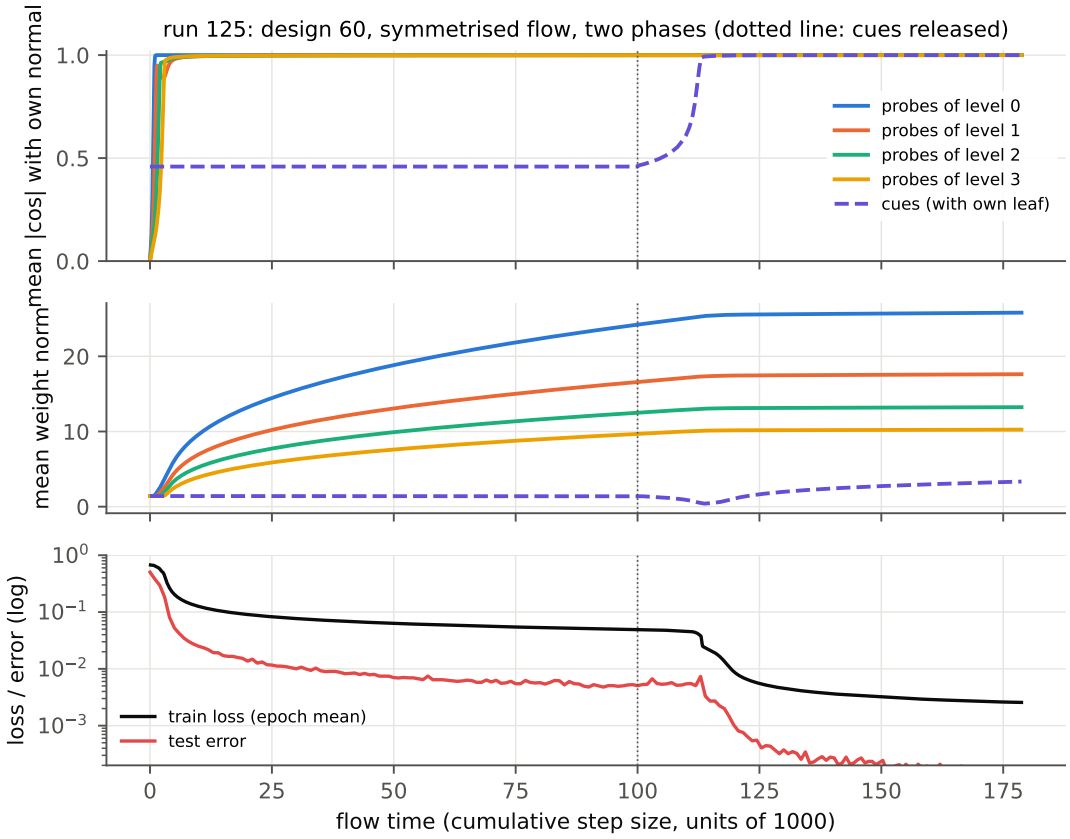


Figure 4: Run 125 (design 60, symmetrised flow, two phases), the reference run. Top: mean $|\cos|$ of the probes with their own routing normal by level, and of the cues with their own leaf normal. Middle: mean weight norm by level (the root's probes grow largest). Bottom: training loss and test error. The dotted line is the release of the cues.

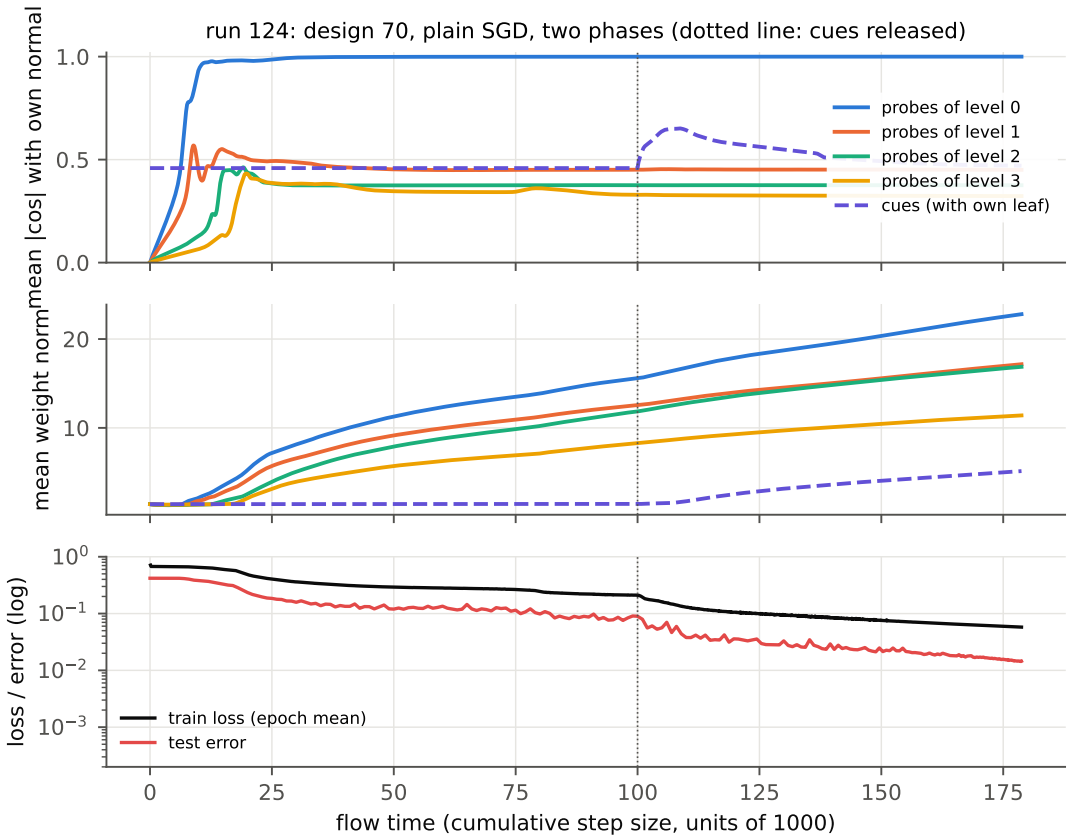


Figure 5: Run 124 (design 70, plain SGD, two phases). The root is captured; the probes of levels 1–3 are decided by the sample and most of them climb to an ancestor, so the plateau is high and the cues, once released, cannot complete the tree.

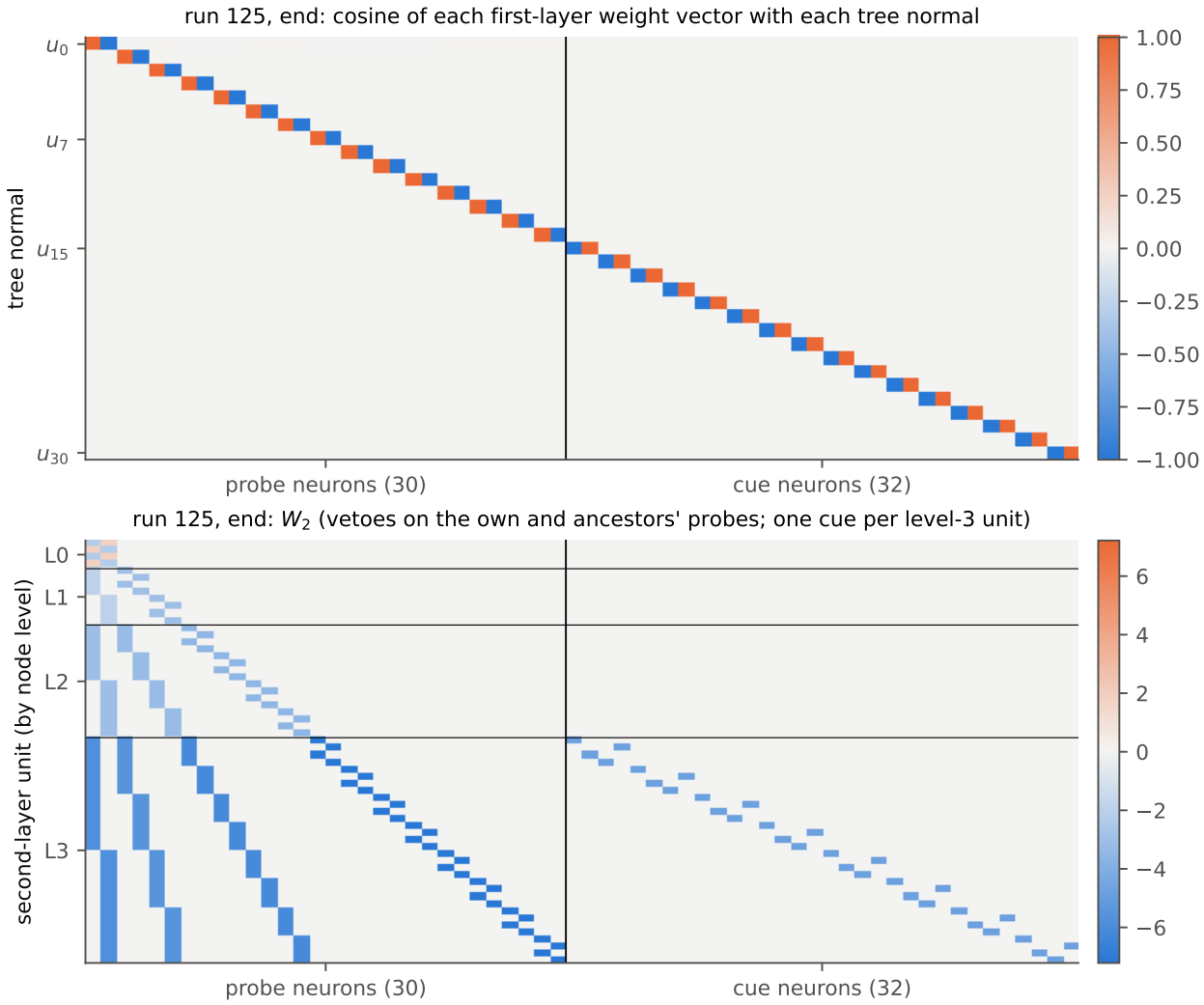


Figure 6: Run 125 at the end. Top: cosine of each first-layer weight vector (columns) with each tree normal (rows): every probe is $\pm u_j$ for its node and every cue $\pm u_m$ for its leaf. Bottom: W_2; the level-3 units veto their own probe and the probes of their ancestors (the diagonal bands on the probe columns), and the cue block has become one cue per unit.

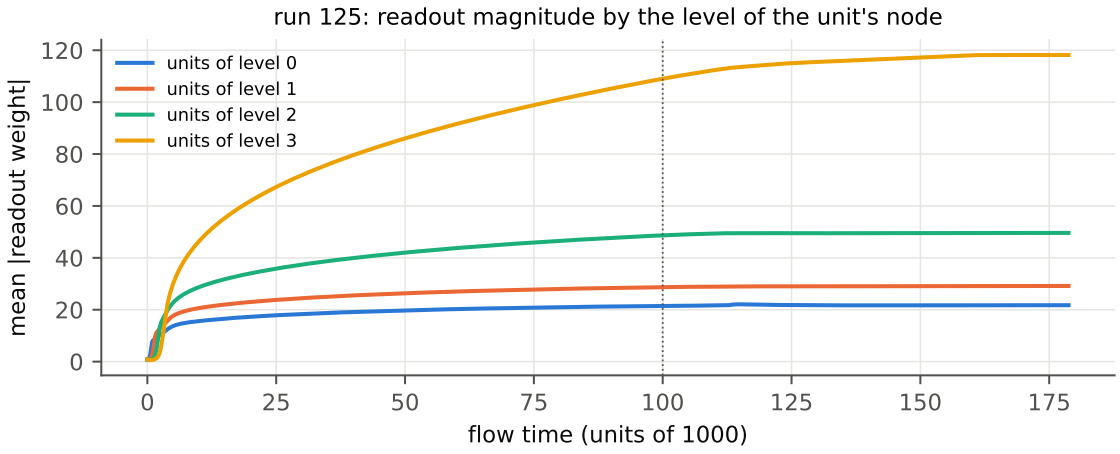


Figure 7: Run 125: mean $|W_{\text{out}}|$ of the units by the level of their node, against flow time. The level-3 units take over once their probes and cues are in place.

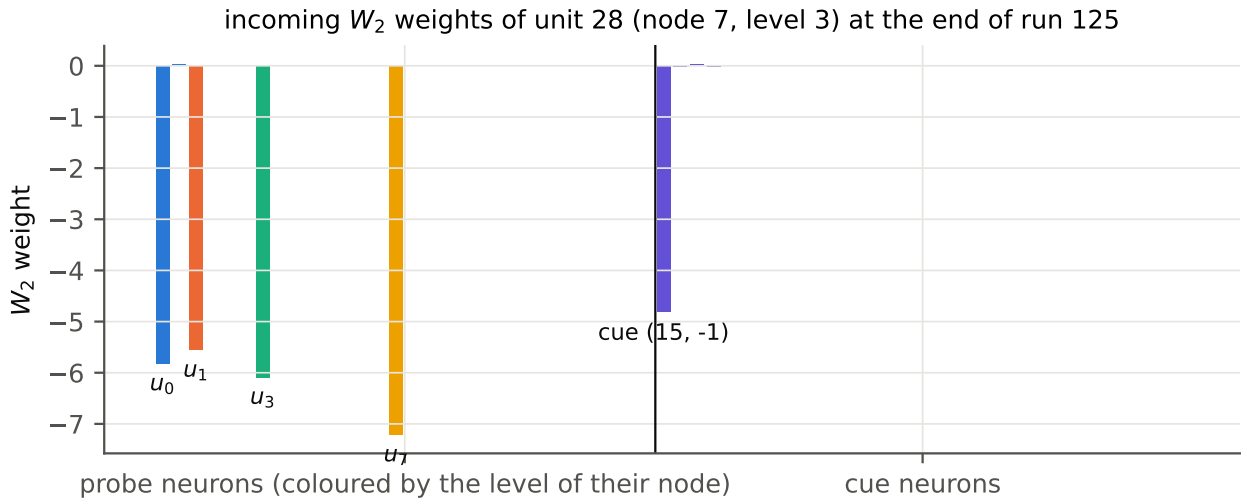


Figure 8: Incoming weights of one level-3 unit (unit 28, node 7) at the end of run 125: the veto chain along its path to the root plus one cue veto, on top of a small positive background on the other cues.

6 Symmetrised gradients: simulating population gradient flow

The group. Let G be generated by the 15 flips F_j (negate u_j, exchange the two subtrees below j) and the label negation N (negate the 16 leaf normals, which negates y); $|G| = 2^{16} = 65\,536$. Each $g \in G$ acts on the parameters by permuting the neurons according to their identities (probe $(j, \pm)$, cue $(m, \pm)$), permuting the units according to theirs (node, block sign, path sign), and flipping the signs of the affected coordinates of W_1 and of W_{out}. The loss is invariant, $L(g \cdot \theta) = L(\theta)$, and both designed initialisations are fixed points, $g \cdot \theta_0 = \theta_0$. Gradient flow on the population loss therefore stays on the fixed-point set forever; finite-sample SGD leaves it at the first step.

The symmetrised flow. We run the same SGD but project every minibatch gradient onto the symmetric subspace: coordinates of $(W_1, W_2, W_{\text{out}})$ are grouped into orbits of G (with signs), each gradient entry is replaced by the signed mean over its orbit, and orbits that contain a coordinate and its own negative are set to zero. This equals the gradient on the minibatch augmented by all 65 536 images of its points, at the cost of one backward pass and a bincount. Of the 7562 coordinates, 203 orbits remain, 157 of them free (Figure 9); a probe's in-span part, for instance, can only move along its own normal and the normals of its ancestors. Sampling noise survives only inside this subspace, so the trajectory is an estimate of the population flow from the symmetric init, with the usual step-size discretisation.

Design 60: SGD and the flow agree. Runs 123 and 125 capture the levels at the same flow times, rotate the cues at the same point and end at the same solution (left panel of Figure 11): the design's pushes are 30–40 times the sampling noise, so the noise is irrelevant.

Design 70: the flow finds what SGD loses. The symmetrised run 126 captures all four levels (epochs 96, 370, 310, 363), every probe on its own node, rotates the cues at epoch 1104 and ends at loss 0.0196 with all 31 normals covered (Figure 10, right panel of Figure 11), where SGD from the same init captured only the root. The weak majority weights of design 70 give pushes only a few times the sampling noise, and under SGD the probes of levels 1–3 are decided by the sample: they climb to the root or to an ancestor. Under the population flow the same wiring is enough. The transient dips of the level-1 and level-2 cosines in Figure 10 are the competition between a node and its ancestors, which the symmetric flow resolves in favour of the node. The lesson is that the mechanism is deterministic and complete, and that the failures of SGD on the weaker init are symmetry breaking by sampling, not a missing ingredient.

7 Why two phases?

Freezing the cues for the first 1000 epochs is a device to keep the lens fixed while the probes learn, and the one-phase runs say what it buys (Figure 12, Figure 13). With the cues free from the start their leaf bias is not

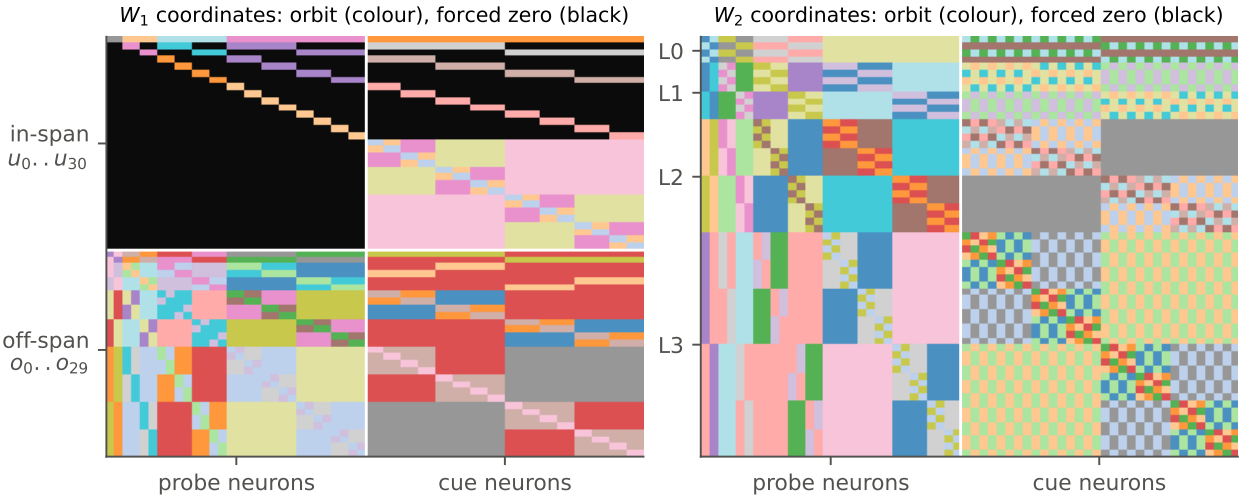


Figure 9: Orbit structure of the parameter coordinates (colour = orbit; colours repeat, black = forced to zero). In W_1 (columns: neurons; rows: coordinates in the basis of the 31 normals and the 30 initial probe directions), a probe's in-span coordinates are free only on its own normal and its ancestors' (the staircase in the top-left block), the cues' in-span coordinates on the leaf normals, and all off-span coordinates are free. In W_2 every block is free, tied across the units of a level.

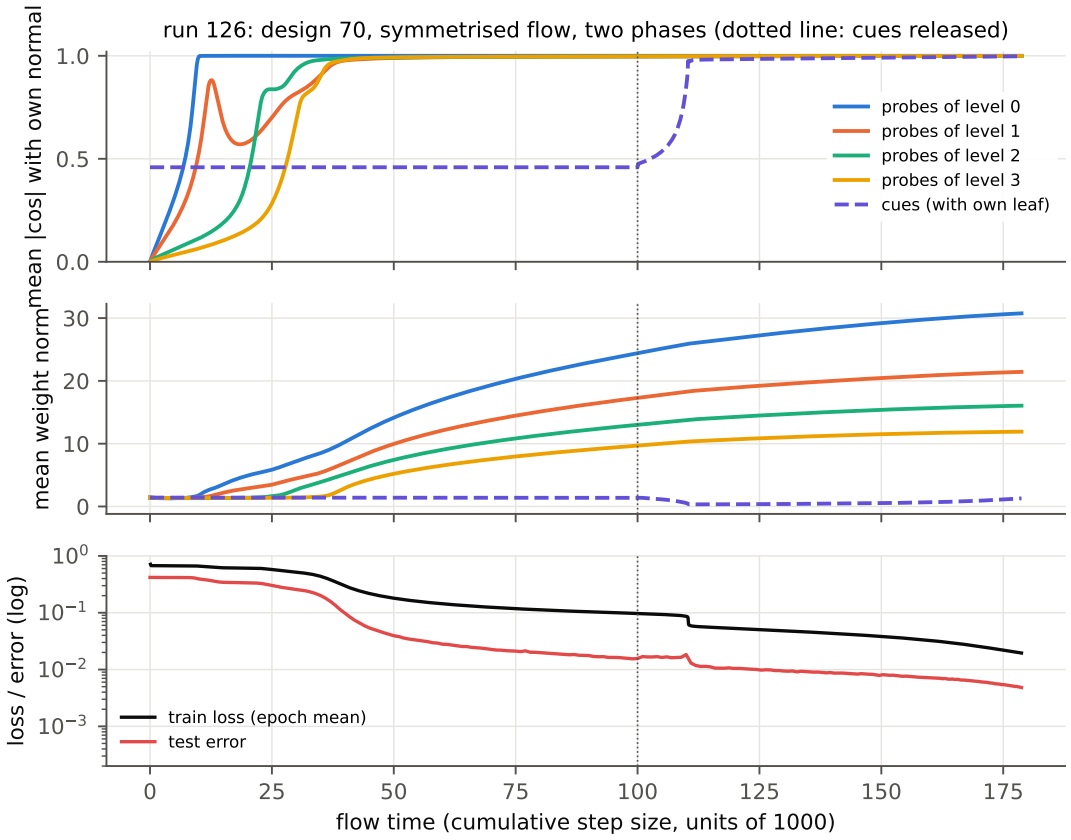


Figure 10: Run 126 (design 70, symmetrised flow, two phases). All four levels are captured during the frozen phase and the cues rotate after their release; compare the SGD run from the same init in Figure 5.

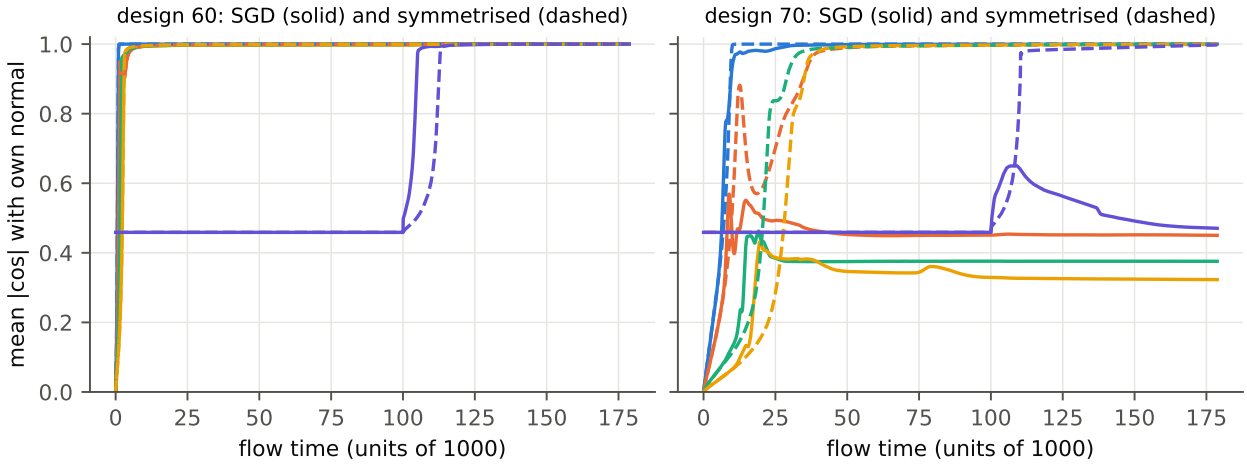


Figure 11: SGD (solid) against the symmetrised flow (dashed), two-phase runs, probe cosines by level and cue cosine (violet). Left: design 60, the two coincide. Right: design 70, SGD loses levels 1–3 to noise-driven competition, the flow captures them all.

protected: the cue neurons are themselves pushed, mostly along v and towards each other, and their cosine with their own leaf normal falls before the probes of the deeper levels are placed. The probes of levels 0–2 are still captured (at similar flow times), but the level-3 probes and the cue identities are not recovered within the run, and the final loss stays an order of magnitude above the two-phase runs: 0.0373 against 0.0026 for design 60 under the flow, 0.0060 against 0.0042 under SGD. The two phases therefore separate two uses of the same cue neurons: first as fixed lenses that teach the probes the routing normals, then as neurons that learn the leaf normals once the leaf cells can be isolated. In a network with many more neurons this separation would be supplied by having different neurons in the two roles; here it is supplied by the schedule.

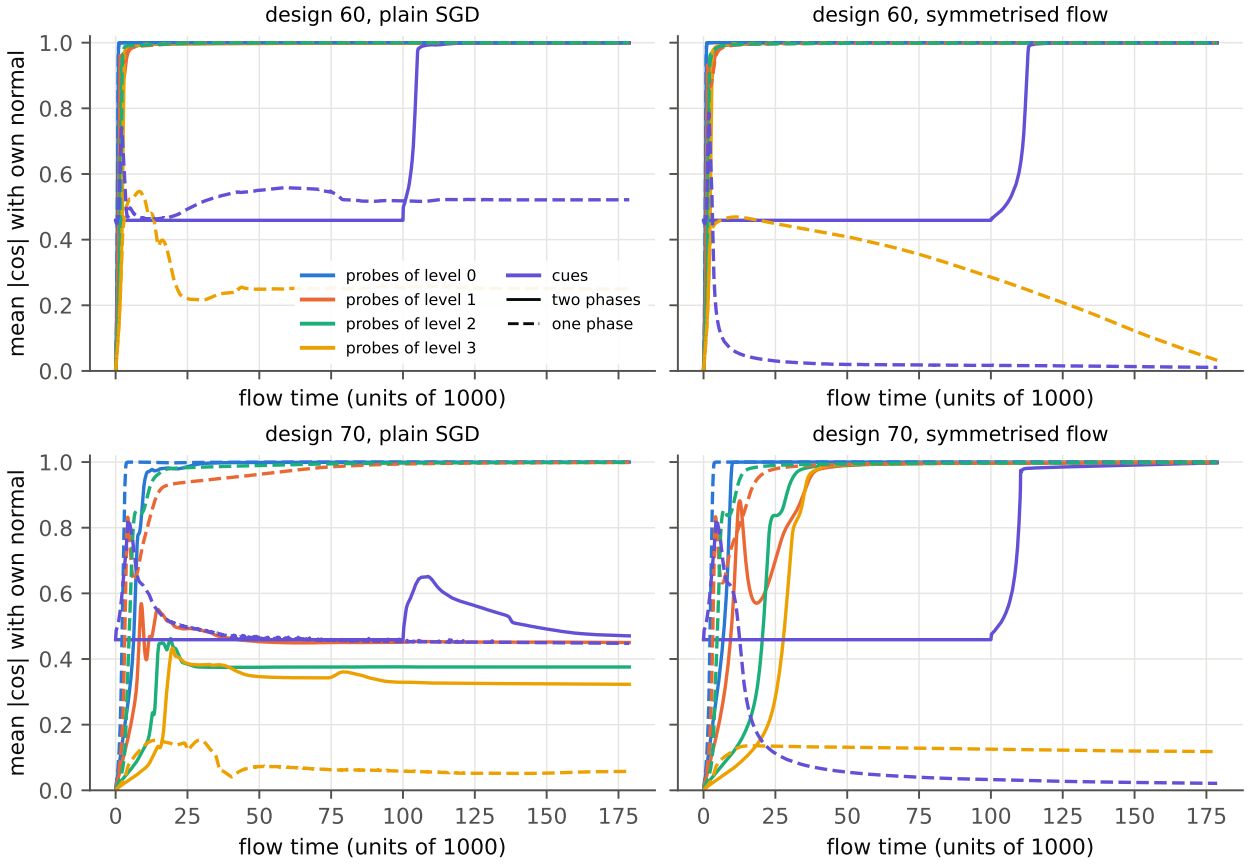


Figure 12: Two phases (solid) against one phase (dashed), by design and optimiser: mean $|\cos|$ of the probes with their own normal by level and of the cues with their own leaf (violet), against flow time.

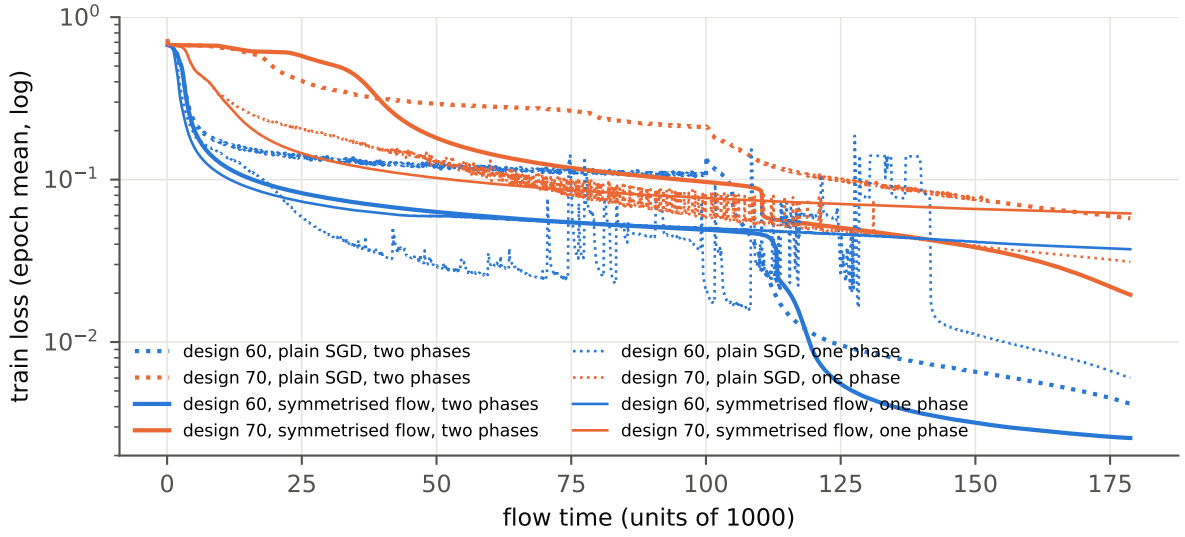


Figure 13: Training loss of the eight runs against flow time (blue: design 60, orange: design 70; solid: symmetrised, dotted: SGD; thick: two phases, thin: one phase). The spikes of the SGD curves during the constant-step phase are step-size instabilities at the full step, absent from the symmetrised runs.

8 What this buys, and where it is going

- **A mechanism, not a correlation.** The lens formula (2) predicts which neuron moves along which normal from the wiring alone, including the order (root first, because the side contrast of a level-l block sums over 2^{4-l} leaves) and the transient ancestor components. The symmetrised runs show the mechanism works without any help from noise.
- **The "cheating" budget is explicit.** The init uses the tree in three places: the cue bias β, the side contrast, and the absence of off-span components in the cues. Each is one scalar of the α parametrisation and has been swept (notebook 2, sections 34–38). The flow restores the third by itself within tens of epochs, the first costs time like $1/\beta$ with no threshold in the population flow, and the second is the one quantity gradient descent does not create from a side-symmetric start.
- **The open theoretical question** is the random-init version of this story: a random first layer has a small component of order 0.1 on every routing normal, a random W_2 reads the two sides of a node unequally by chance, and the loop "probe component on $u_j \to$ cue side contrast on $j \to$ probe push on j" is a per-node instability of the symmetric state whose growth rates should be ordered by level. Making that precise, and testing it with large-batch GD from random inits, is the next line of work.

Where things are in the repository. Code: `src/layer_lenses/alpha_init.py` (the α parametrisation of the inits), `symmetric_flow.py` (orbits and projection), `run_analysis.py` and `path_analysis.py` (trajectory and path tools). Runner: `results/designed_init/run_symmetric_gd.py` runs both plain SGD (`--no-symmetrise`) and the symmetrised flow; the eight runs here are launched by `scripts/launch_clean_runs_long.sh`. Notebooks: `notebooks/hierarchy_design_runs_2.ipynb` (one section per run, with findings) and `notebooks/two_layer_network_anatomy.ipynb` (choose a run and an epoch). Notes: `notes/session_log.md` is the chronological record; this handout, its figure script `make_figures.py` and the generated `facts.tex` are in `notes/handout/`.